



LEMBAR KERJA-01

Sistem Otomatis Deteksi Anomali Data Cuaca Open-Meteo untuk Smart Farming Berbasis MLOps MLOPS TIF-B 2026

Nama:

Muhammad Ghazy Humaidi (245150200111071)

Dosen Pengampu:

Rizal Setya Perdana, S.Kom., M.Kom., Ph.D.



**Program Studi Teknik Informatika
Jurusan Teknik Informatika**

2. Identifikasi Masalah & Domain

2.1 Domain Proyek

Proyek ini berfokus pada domain **Internet of Things (IoT) dan Smart Agriculture (Pertanian Pintar)**. Secara spesifik, proyek ini menangani manajemen data metrik lingkungan yang dihasilkan oleh infrastruktur *Smart Farming* otomatis yang dideploy di lapangan.

2.2 Machine Learning Task

Pendekatan *Machine Learning* yang diterapkan pada proyek ini adalah **Multiclass Classification** untuk deteksi dan diferensiasi anomali data. Model akan mengklasifikasikan aliran data numerik *time-series* dari sensor secara *real-time* ke dalam 3 kelas keputusan yang *actionable*:

Class 0: Normal State Pembacaan sensor berada dalam rentang operasional yang wajar dan berkorelasi secara logis dengan kondisi cuaca aktual dari *Open-Meteo API*.

Class 1: Hardware Fault Anomaly Indikasi kegagalan perangkat keras di lapangan (seperti kabel terputus, sensor terendam, atau *battery drop*). Ditandai dengan pola data tidak realistis seperti nilai konstan/mampet (*stuck value*), lonjakan ekstrem tak masuk akal (*spiking*), atau pembacaan *out-of-bounds*.

Class 2: Extreme Environmental Anomaly Indikasi perubahan kondisi lingkungan yang nyata dan ekstrem (seperti gelombang panas mendadak atau kekeringan lahan parah). Data divalidasi dan terkonfirmasi relevan dengan tren data iklim eksternal.

Klasifikasi multiclass ini dipilih dengan alasan agar model dapat memberikan respon yang tepat pada sistem aktuator *Smart Farming*. Jika terdeteksi Class 1, sistem akan mengabaikan data dan mengirimkan sinyal pemeliharaan ke tim teknisi tanpa mengganggu operasi penyiraman. Sebaliknya, jika terdeteksi Class 2, sistem akan memicu tindakan mitigasi otomatis (seperti mengaktifkan pompa irigasi) untuk melindungi tanaman.

2.3 Latar Belakang Masalah (WHY)

Dalam implementasi dan pengembangan infrastruktur *Smart Farming* yang dirancang untuk beroperasi secara otomatis dari waktu ke waktu, keandalan data dari *node* sensor menjadi sangat krusial. Perangkat keras yang ditempatkan di luar ruangan sangat rentan terhadap degradasi fisik, gangguan cuaca ekstrem, fluktuasi daya, atau gangguan konektivitas jaringan.

Seringkali, pembacaan data yang tidak normal dianggap sebagai data yang benar oleh sistem aktuator. Jika anomali ini gagal diidentifikasi, sistem otomatisasi akan membuat keputusan fatal, seperti terus-menerus menyalakan pompa air yang dapat merusak lahan.

Oleh karena itu, penyelesaian menggunakan *Traditional ML* statis tidak akan cukup karena parameter lingkungan akan terus bergeser seiring pergantian musim (kemarau ke penghujan) yang menyebabkan terjadinya *concept drift* dan *data drift*. Dibutuhkan sistem dengan pendekatan MLOps (*production-ready*) yang mampu mendeteksi anomali sensor secara langsung (*real-time*), serta memiliki mekanisme *continual learning* untuk beradaptasi dengan perubahan pola data lingkungan dan perangkat keras selama masa operasional berbulan-bulan.

2.4 Pengguna Target (WHO)

Sistem MLOps ini dikembangkan untuk memberikan nilai bisnis dan operasional kepada:

1. **Engineer Infrastruktur & Tim Peneliti (Administrator Sistem):** Memanfaatkan metrik dan dasbor monitoring untuk memantau kesehatan *hardware* (*node* sensor) secara terpusat tanpa perlu melakukan observasi lapangan (*field check*) secara manual setiap hari.
2. **Pengelola Lahan / Pengguna Akhir (Petani Modern):** Mendapatkan sistem otomasi yang lebih persisi, aman dari kegagalan sistematis (*fault-tolerant*), dan peringatan dini apabila terdapat perangkat di lapangan yang memerlukan pergantian atau *maintenance*.

3. Karakteristik Data Dinamis

3.1 Sumber Data dan Sistem Pengumpulan

Proyek ini sepenuhnya memanfaatkan **Open-Meteo API** sebagai sumber data tunggal (*single primary data source*). Open-Meteo merupakan layanan *Open-Source Weather API* non-komersial yang menyediakan data cuaca dan metrik agrikultur secara *real-time* maupun historis tanpa memerlukan *API key* khusus. Beberapa rasionalisasi utama pemilihan Open-Meteo API meliputi:

1. **Aksesibel & Bebas Biaya (*Open-Access*):** Sangat ideal untuk pengembangan dan pembelajaran MLOps karena tidak memiliki batasan kuota (*rate limit*) yang ketat serta tidak memerlukan biaya langganan, sehingga menjamin keberlanjutan *pipeline* otomatisasi.
2. **Ketersediaan Data Agrikultur yang Kaya:** Menyediakan variabel meteorologi lengkap yang relevan untuk *Smart Agriculture*, seperti suhu udara (*temperature_2m*), kelembaban relatif (*relative_humidity_2m*), curah hujan (*precipitation*), kelembaban tanah pada berbagai kedalaman (*soil_moisture_0_to_7cm*), dan radiasi sinar matahari (*shortwave_radiation*).
3. **Mendukung Simulasi Data Dinamis & Real-time:** API ini menyediakan *endpoint* data prakiraan (*forecast*) dan data terkini (*current weather*) yang terus di-update setiap jam berdasarkan model analisis iklim global. Hal ini memungkinkan sistem MLOps untuk melakukan penarikan data secara berkala (*periodic micro-batch fetching*) tanpa perlu mengelola perangkat keras (*hardware sensor*) secara fisik.

3.2 Modalitas Data

Modalitas data yang digunakan dalam proyek ini adalah **Tabular Time-Series**. Data bersumber dari respons API berformat JSON yang diekstrak dan ditransformasikan ke dalam struktur data terstruktur (tabel/DataFrame) secara berkala. Karakteristik utama dari modalitas ini meliputi:

1. **Representasi Temporal (*Time-Series*):** Setiap baris data terikat secara ketat oleh indeks waktu (*timestamp* jam-jaman). Urutan kronologis data sangat krusial karena model membutuhkan pola tren historis untuk memprediksi dan mengklasifikasikan anomali cuaca secara tepat.
2. **Atribut Numerik Kontinyu:** Fitur utama berupa variabel meteorologi dan agrikultur kontinyu, seperti suhu udara (*temperature_2m*), kelembaban relatif (*relative_humidity_2m*), presipitasi (*precipitation*), kelembaban tanah (*soil_moisture*), dan radiasi matahari (*shortwave_radiation*).
3. **Atribut Spasial:** Koordinat lokasi (*latitude, longitude, elevation*) berfungsi sebagai penanda area lahan pertanian yang dipantau.

3.3 Frekuensi Update (*Update Frequency*) Sistem akan menerapkan mekanisme **Periodic Update**. Data tidak diambil secara *live streaming* per detik untuk menghindari *bottleneck* komputasi, melainkan di-*fetch* secara *micro-batch* setiap **1 jam sekali**. Frekuensi ini dinilai paling optimal untuk menangkap tren perubahan cuaca atau anomali lingkungan di lahan pertanian tanpa membebani infrastruktur jaringan maupun *storage*.

3.4 Potensi *Data Drift* dan *Concept Drift*

Mengingat siklus operasional sistem MLOps ini berjalan secara kontinu melintasi pergantian bulan dan musim, model *Machine Learning* akan terekspos pada perubahan distribusi data iklim. Terdapat dua jenis *drift* utama yang diantisipasi:

1. **Data Drift (Covariate Shift):** Dalam project ini akan memiliki contoh sebagai berikut: Pada musim penghujan, batas atas suhu harian berada pada kisaran 26°C. Namun, saat memasuki musim kemarau, distribusi data *input* bergeser ke rentang 32°C–36°C. Model yang hanya dilatih menggunakan data musim hujan akan mengalami penurunan tingkat kepercayaan (*confidence score*) saat menerima data musim kemarau karena belum pernah mempelajari rentang nilai tersebut. *API Ingestion Drift*: Adanya perubahan format atau pola keterlambatan (*latency*) dari respons JSON Open-Meteo API seiring pembaruan versi server mereka.
2. **Concept Drift:** Dalam project ini akan memiliki contoh sebagai berikut: Pembacaan suhu udara di angka 35°C pada puncaknya musim hujan merupakan fenomena tidak lazim yang masuk ke dalam **Class 2 (Extreme Environmental Anomaly)**. Namun, angka 35°C di siang hari pada puncak musim kemarau merupakan kondisi wajar yang tergolong **Class 0 (Normal State)**. Jika model tidak diperbarui secara berkala, model statis akan terus menghasilkan *False Positive*.

Kehadiran *Data Drift* dan *Concept Drift* secara alami pada data iklim inilah yang menjadi alasan utama mengapa sistem ini membutuhkan *monitoring* berbasis MLOps dan *Continuous Training* (CT) otomatis.

4. Strategi Continual Learning (CL/CT)

Mengingat operasional infrastruktur *Smart Farming* pada ICN Lab dijadwalkan berjalan secara kontinu hingga pertengahan tahun 2027, sistem tidak bisa bergantung pada satu model statis. Strategi *Continual Learning* / *Continuous Training* (CL/CT) dirancang untuk memastikan model deteksi anomali dapat beradaptasi secara otomatis terhadap perubahan cuaca, pergantian musim, dan degradasi kondisi *hardware* di lapangan.

4.1 Mekanisme Fetch Data Berkala (Data Ingestion)

Sistem tidak memerlukan intervensi manual dari pengguna untuk mengumpulkan data baru. Proses *data ingestion* dan pengorganisasian alur kerja (*data pipeline*) diotomatisasi penuh menggunakan **Apache Airflow** sebagai *orchestration engine* utama.

1. Jadwal Fetching & Eksekusi DAG (*Hourly Interval*):

- Penarikan data diatur melalui *Directed Acyclic Graph* (DAG) pada Apache Airflow yang dijadwalkan berjalan otomatis setiap 1 jam sekali (*schedule_interval='@hourly'*).
- *Operator* khusus pada Airflow akan mengeksekusi *HTTP GET Request* ke *endpoint* Open-Meteo API untuk menyedot metrik meteorologi dan agrikultur terbaru.

2. Rasionalisasi Pemilihan Apache Airflow:

- **Observabilitas & Monitoring Visual:** Airflow menyediakan *Web UI* terpusat untuk memantau status eksekusi *pipeline* secara *real-time*, memudahkan *debugging*, serta melihat histori *data ingestion*.
- **Manajemen Kegagalan (*Robustness & Auto-Retry*):** Apabila *endpoint* Open-Meteo API mengalami *timeout* atau gangguan jaringan, Airflow memiliki fitur *automatic retry* dengan mekanisme *exponential backoff* sehingga data tidak akan hilang (*data loss*).
- **Skalabilitas Pipeline:** Memudahkan integrasi tingkat lanjut pada tahap berikutnya (seperti pemicuan otomatis *task* pembersihan data dan *task retraining* model dalam satu rantai DAG yang terstruktur).

3. Penyimpanan & Versi Data (*Data Logging & Versioning*):

- Respons JSON dari API diekstrak menjadi bentuk tabular, lalu disimpan (*appended*) ke dalam database/storage terpusat.
- Perubahan histori *dataset* dicatat menggunakan *Data Version Control* (DVC) untuk menjamin *reproducibility* data.

4.2 Pemicu Retraining Otomatis (Retraining Triggers)

Untuk menjaga keseimbangan antara performa prediksi model dan efisiensi biaya komputasi, proses pelatihan ulang (*retraining*) tidak dilakukan secara impulsif setiap kali data jam-jaman masuk. Sistem MLOps menerapkan pendekatan **Hybrid Triggers** yang mengombinasikan tiga logika pemicu utama:

1. **Scheduled Trigger (Retraining Berbasis Waktu/Kalender):** *Pipeline retraining* dieksekusi secara otomatis oleh Airflow DAG setiap **2 minggu sekali**. Hal tersebut

bertujuan sebagai tindakan preventif untuk menangkap pergeseran iklim alami yang terjadi secara perlahan (seperti transisi dari musim hujan ke musim kemarau). Model akan dilatih menggunakan akumulasi *sliding window* data 30 hari terakhir dari Open-Meteo API.

2. **Data Drift & Concept Drift Trigger (Retraining Berbasis Distribusi Data):** Layanan *monitoring* (menggunakan **Evidently AI**) mengevaluasi distribusi fitur meteorologi harian secara otomatis. Jika pengujian statistik (*Wasserstein Distance* atau *KS-Test*) pada fitur krusial (seperti *temperature_2m* dan *soil_moisture*) mencatat nilai pergeseran melebihi ambang batas toleransi (*drift threshold* > 0.1), sistem akan langsung mengirim sinyal pemicu *retraining* darurat (*event-driven trigger*) tanpa menunggu jadwal 2 mingguan.
3. **Performance Degradation Trigger (Retraining Berbasis Metrik Performa):** Evaluasi terhadap perkiraan F1-Score model di lingkungan produksi. Jika *monitoring dashboard* mencatat terjadinya penurunan performa klasifikasi di mana **F1-Score berada di bawah 0.80** (misalnya model sering salah membedakan *Class 0: Normal* dengan *Class 2: Extreme Weather* akibat perubahan pola cuaca mendadak), sistem CI/CD akan ter-pemicu secara otomatis untuk melakukan *re-labeling* dan *retraining* ulang.

4.3 Mekanisme Validasi Model (Shadow Deployment) Agar proses otomatisasi tidak merusak sistem yang sedang berjalan di *production*, setiap model baru hasil *retraining* otomatis tidak akan langsung menggantikan model lama. Model baru harus melewati evaluasi metrik pada *hold-out dataset*. Jika performanya lebih baik, model baru akan di-*deploy* secara *seamless* melalui sistem API.

5. Kriteria Sukses

Kriteria keberhasilan dari proyek MLOps ini dievaluasi melalui dua dimensi utama: metrik teknis dari performa model, dan metrik bisnis yang merepresentasikan dampak nyata terhadap operasional *Smart Farming*. Kedua metrik ini saling berkesinambungan.

5.1 Metrik Teknis (Technical Metrics)

1. **F1-Score (> 85%):** Karena deteksi anomali seringkali berhadapan dengan data yang tidak seimbang, *F1-Score* digunakan sebagai metrik utama dibandingkan sekadar akurasi. Sistem harus meminimalkan *False Negatives* dan *False Positives*.
2. **Latency Inference (< 500 ms):** Prediksi anomali harus dilakukan secara *real-time* atau mendekati *real-time*. Waktu yang dibutuhkan API untuk menerima aliran data sensor, memproses model, dan mengembalikan hasil prediksi tidak boleh melebihi 500 milidetik per *request*.
3. **Availability / Uptime API (99.9%):** Layanan model yang di-*deploy* harus selalu tersedia untuk menerima data dari *node* IoT di lapangan yang terus menyala 24/7.

5.2 Metrik Bisnis & Dampak Operasional (Business Metrics)

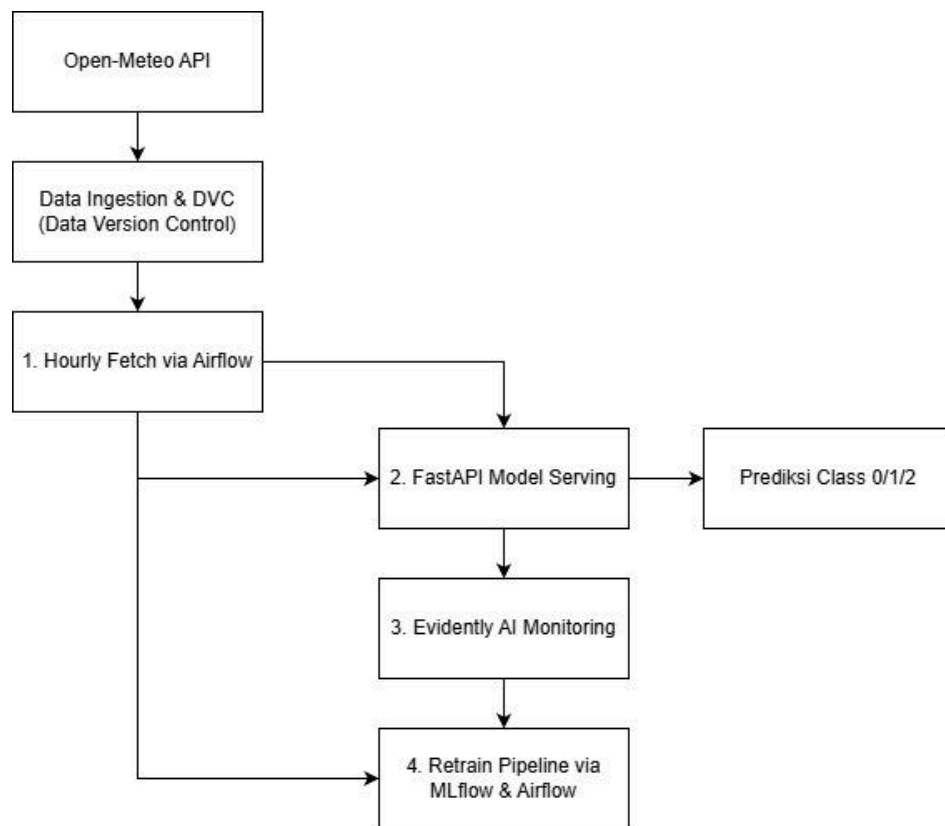
1. **Efisiensi Operasional (Pengurangan Pengecekan Manual):** Target utama adalah mengurangi frekuensi inspeksi perangkat keras (*hardware sensor*) ke lapangan secara

manual oleh tim pengelola minimal sebesar **40%**. Sistem hanya akan memanggil teknisi jika dasbor mendeteksi anomali persisten yang tervalidasi.

2. **Risk & Cost Reduction (Pencegahan Kegagalan Aktuator):** Mengurangi risiko aktuator (seperti pompa air otomatis) menyala di waktu yang salah akibat data sensor yang cacat (misal: sensor rusak mengira tanah kering padahal banjir). Hal ini menghemat biaya listrik, air, dan mencegah kerusakan tanaman (*crop failure*).
3. **Adaptability (Pengurangan Downtime Kalibrasi):** Dengan adanya *Continuous Training* otomatis, waktu *downtime* sistem untuk kalibrasi manual saat pergantian musim (kemarau ke hujan) ditargetkan turun hingga mendekati nol, karena model beradaptasi secara otomatis.

6. Arsitektur Pipeline

Arsitektur MLOps ini terbagi menjadi 4 tahapan eksekusi otomatis yang saling terhubung dan dapat dilihat melalui tabel berikut ini



6.1 Diagram Alur End-to-End (Data → Training → Deployment → Monitoring)

Tahap 1: Data Ingestion & Data Versioning (Setiap 1 Jam)

- **Cara Kerja:** Apache Airflow mengeksekusi DAG harian/jam-jaman. Airflow mengirim *HTTP GET Request* ke Open-Meteo API.
- **Proses:**

1. Data JSON dari API ditarik, diekstrak variabelnya (*temperature_2m*, *precipitation*, *soil_moisture*), dan diubah menjadi *DataFrame* Pandas.
2. Data divalidasi (memastikan tidak ada kolom hilang). Data baru lalu digabungkan (*appended*) ke file *dataset_current.csv*.
3. **DVC (Data Version Control)** mencatat *hash MD5* dari file data terbaru. Ini memastikan kita bisa kembali (*rollback*) ke versi data jam tertentu jika ada masalah.

Tahap 2: Continuous Training & Model Registry (Saat Trigger Aktif)

- **Cara Kerja:** Jika Airflow menerima sinyal *Retraining Trigger* (jadwal 2 mingguan atau alarm dari Evidently AI), Airflow akan menjalankan *Retraining DAG*.
- **Proses:**
 1. Airflow memanggil *script* Python untuk *preprocessing*, *feature scaling*, dan melatih model (antara XGBoost / Random Forest) dengan data 30 hari terakhir.
 2. **MLflow Tracking** mencatat semua parameter (*hyperparameter*), metrik evaluasi (*F1-Score*, *Confusion Matrix*), dan menyimpan *file* artefak model (*model.pkl*).
 3. Model yang baru dilatih didaftarkan ke **MLflow Model Registry** dengan status Staging (Model Challenger).

Tahap 3: Model Serving & Deployment (Real-Time Inference)

- **Cara Kerja:** Model dipaketkan ke dalam **FastAPI** yang berjalan di dalam **Docker Container**.
- **Proses:**
 1. FastAPI membuka *endpoint* REST API (misal: POST /predict).
 2. Saat ada data baru dari Open-Meteo API, sistem memanggil *endpoint* tersebut.
 3. FastAPI memuat model dari MLflow, melakukan inferensi cepat (< 100 ms), dan mengembalikan respons JSON: *{"prediction": "Class 0", "confidence": 0.94}*.
 4. Dalam strategi *Shadow Deployment*, FastAPI mengeksekusi prediksi dari model *Champion* (Lama) dan model *Challenger* (Baru) secara beriringan untuk membandingkan akurasi di latar belakang.

Tahap 4: Monitoring, Observability & Feedback Loop

- **Cara Kerja:** Sistem tidak dilepas begitu saja, melainkan terus diawasi secara *real-time*.
- **Proses:**
 1. Setiap input data dari Open-Meteo API beserta hasil prediksi dari FastAPI dicatat ke dalam *Log Database*.
 2. Setiap hari, **Evidently AI** membandingkan data 24 jam terakhir dengan data *reference* saat *training*.
 3. Evidently menghitung statistik *Data Drift* dan *Concept Drift*.

4. **Feedback Loop:** Jika Evidently menemukan *drift score* > 0.1, Evidently mengirimkan panggilan *Webhook* / *API Call* ke Apache Airflow untuk mengaktifkan kembali **Tahap 2 (Continuous Training)** secara otomatis.

6.2 Pemilihan Tech Stack

Komponen / Layer	Teknologi Utama	Cara Kerja
Language & Data Processing	Python & Pandas	Bahasa pemrogram utama dan <i>library</i> manipulasi data tabular dari JSON API menjadi <i>DataFrame</i> .
Orchestration & Workflow	Apache Airflow	"Sutradara" utama yang mengatur eksekusi <i>scheduler</i> jam-jaman, pengulangan otomatis jika <i>API down</i> , serta urutan alur kerja <i>ingestion</i> dan <i>retraining</i> .
Data Versioning	DVC (Data Version Control)	"Git untuk Data". Mencatat versi file CSV/Database agar setiap model yang dilatih bisa dilacak data historis asalnya (<i>reproducibility</i>).
Experiment Tracking & Registry	MLflow	Menyimpan <i>history</i> percobaan <i>training</i> , mencatat metrik <i>F1-Score</i> , serta menjadi repositori tempat menyimpan file model <i>.pkl</i> .
Model Serving (API)	FastAPI	<i>Framework</i> Python berlatensi rendah untuk membuat REST API yang menerima input data cuaca dan mengembalikan hasil klasifikasi (Class 0/1/2).
Containerization	Docker	Mengemas aplikasi FastAPI, Python, dan <i>library</i> pendukung ke dalam satu <i>container</i> terisolasi agar bisa dijalankan di <i>server</i> mana saja tanpa masalah bentrok dependensi (<i>environment consistency</i>).
Monitoring & Drift Detection	Evidently AI	"Dokter" pemantau kesehatan data. Mengukur pergeseran distribusi (<i>Data & Concept Drift</i>) dan memicu

		<i>retraining</i> otomatis jika performa memburuk.
Visual Dashboard	Grafana	Menampilkan grafik visual harian terkait status cuaca, tren prediksi model, dan metrik <i>drift</i> agar mudah dipantau oleh tim engineer.

7. Lampiran (Proof of Concept)

Bukti bahwa data dapat dimanipulasi dan cocok untuk studi kasus yang diangkat dapat dilihat melalui link google collab berikut ini

 **Proof Of Concept - Muhammad Ghazy Humaidi**